

백엔드·데이터·AI 인프라 포트폴리오

분산 시스템의 실패 복구와 검증 자동화를 설계하는 백엔드 엔지니어입니다. AI 검색·CDC·개인화 시스템까지 같은 기준으로 확장했습니다.

Spring Boot

Python API

Kafka

PostgreSQL

Redis/RabbitMQ

RAG/LLM

대표 프로젝트

Java 백엔드 · 정합성 · 동시성 · AI 검색

직무별 확장 프로젝트

LLM 플랫폼 · CDC · Python 개인화

01 StockRush

분산 커머스 / 주문 상태 수렴 / 이벤트 기반 백엔드

Spring Boot

Kafka

PostgreSQL

04 AI Gateway

LLM Gateway / 라우팅 / 비용 보호 / 캐시 / 폴백 / 관측

Java 21

Spring Boot WebFlux

Redis

03 Member Event Consistency

동시성 / 업무 불변식 / DB guard / Redis-RabbitMQ 비교

Spring Boot

PostgreSQL

Redis

05 CDC Data Platform

CDC / Kafka / Lakehouse / Lineage / Replay

Spring Boot

PostgreSQL

Debezium

02 Enterprise Policy RAG

AI 백엔드 / RAG / 권한 검색 / 운영 관측

Python API

React/Vite

PostgreSQL/pgvector

06 Fashion Personalization Platform

Python API / 커머스 개인화 / 추천 batch / 이벤트 처리

Python 3.11+

optional FastAPI

pytest

01 StockRush

분산 커머스 / 주문 상태 수렴 / 이벤트 기반 백엔드

대표 프로젝트

한정 재고 주문의 부분 실패와 중복 처리를 Saga, Transactional Outbox, 멍든 소비자로 제어하고 주문·재고·결제·조회 모델의 최종 상태 수렴을 장애 시나리오로 검증했습니다.

문제 부분 실패와 재시도 뒤 주문·재고·결제·조회 상태가 어긋나는 문제
구현 Saga + Outbox + 멍든 consumer + 재고 행 잠금 + JWT Gateway
검증 중복 이벤트 2회→outbox 1건, 늦은 승인 무시, relay retry 5회→FAILED assertion

주문 요청

Saga-Outbox

상태 수렴

Spring Boot

Kafka

PostgreSQL

04 AI Gateway

LLM Gateway / 라우팅 / 비용 보호 / 캐시 / 폴백 / 관측

직무별 확장

OpenAI 호환 WebFlux 진입점에 API key 인증, tenant별 할당량·예산, exact-semantic cache, 정책 routing, circuit breaker-fallback, 요청 추적을 단일 pipeline으로 구현했습니다.

문제 서비스마다 중복되는 LLM 정책과 provider 장애 대응
구현 API key 인증 + quota/budget + two-stage cache + resilient routing
검증 8단계 policy pipeline, 5개 routing mode, JSON-SSE 및 인증 오류 시나리오

정책 적용

routing-fallback

trace-cost

Java 21

Spring Boot WebFlux

Redis

03 Member Event Consistency

동시성 / 업무 불변식 / DB guard / Redis-RabbitMQ 비교

대표 프로젝트

첫 로그인 보상·쿠폰 발급·포인트 차감의 동시성 불변식을 PostgreSQL 제약과 조건부 갱신으로 지키고, Redis lock-RabbitMQ 단일 worker-Outbox 기록을 시나리오별로 비교했습니다.

문제 동일 회원 이벤트의 중복 보상, 쿠폰 초과 발급, 포인트 음수 잔액
구현 DB 제약·조건부 갱신 + Redis lock + RabbitMQ worker + Outbox
검증 8→1 reward, 2×60→balance 40, capacity 3에서 3 issued-5 rejected assertion

불변식 정의

전략 비교

동시성 검증

Spring Boot

PostgreSQL

Redis

05 CDC Data Platform

CDC / Kafka / Lakehouse / Lineage / Replay

직무별 확장

PostgreSQL 변경을 Debezium-Kafka로 수집하고, 멍든 ledger와 retry/DLQ/replay를 거쳐 MinIO-Iceberg-mart까지 이어지는 lineage를 검증했습니다.

문제 CDC 중복 replay, sink 실패, source-to-mart 추적 단절
구현 source metadata event ID + 멍든 ledger + retry/DLQ/replay + local Iceberg
검증 Kafka 중단·재시작-schema evolution-mart lineage 시나리오 검증

CDC 수집

ledger-replay

Iceberg-mart

Spring Boot

PostgreSQL

Debezium

02 Enterprise Policy RAG

AI 백엔드 / RAG / 권한 검색 / 운영 관측

대표 프로젝트

권한이 없는 문서의 검색 노출과 근거 없는 답변을 막기 위해 검색 전 권한 필터, 인용·거절 응답, 질의 비용·지연 추적을 갖춘 사내 정책 RAG를 구현했습니다.

문제 권한이 없는 문서가 검색되거나 근거가 부족한 답변이 생성되는 문제
구현 검색 전 권한 필터 + citation/refusal + query log-eval
검증 pytest-web build-static smoke로 API와 운영 화면의 기본 경로 반복 검증

권한 확인

검색·근거

인용·거절

Python API

React/Vite

PostgreSQL/pgvector

06 Fashion Personalization Platform

Python API / 커머스 개인화 / 추천 batch / 이벤트 처리

직무별 확장

상품 특성과 사용자 행동 이벤트를 개인화 profile과 추천 snapshot으로 변환하고, 중복·실패 이벤트 격리와 freshness 운영 지표를 갖춘 Python 백엔드를 구현했습니다.

문제 중복·실패 이벤트가 개인화 profile과 추천 snapshot을 오염하는 문제
구현 event processor + deterministic ranking + batch snapshot + admin report
검증 동일 key 중복 차단, 3회 실패→DLQ, 6개 상품 fixture와 snapshot freshness assertion

행동 이벤트

profile-ranking

snapshot-report

Python 3.11+

optional FastAPI

pytest

주문 정합성, 권한 기반 검색, 동시성 제어, LLM 정책, CDC lineage, 개인화 추천을 각각 독립된 실행·검증 흐름으로 구현했습니다.



상태·불변식 모델링

복잡한 동시 처리 문제를 상태 전이와 업무 불변식으로 분해하고 경계마다 보호 장치를 선택했습니다.

불변식 정의

보호 장치 선택

동시성 검증

CODE OrderSagaEventHandler · SqlPointSpendRepository

- 주문 Saga와 outbox로 서비스 간 상태 수렴
- DB unique/check/conditional update로 최종 불변식 보호

StockRush

Member Event Consistency



실패 복구 설계

실패를 숨기지 않고 retry, DLQ, replay, fallback처럼 추적하고 다시 처리할 수 있는 상태로 남겼습니다.

실패 상태 기록

복구 경로 실행

수렴 확인

CODE OutboxRelayService · FallbackChain · ReplayRequestService

- Kafka/outbox 중단 뒤 재발행과 먹등 재처리
- LLM fallback, CDC replay, 이벤트 requeue 경로

StockRush

AI Gateway

CDC Data Platform

Fashion Personalization



보안·정책 경계

인증, 권한, 할당량과 provider 정책을 도메인 로직 밖의 명시적인 진입 경계에서 적용했습니다.

신뢰 경계 설정

정책 적용

권한 검증

CODE SecurityConfig · RetrievalService · GatewayPipeline

- OIDC/JWT Gateway와 리소스 소유권 검사
- 검색 전 권한 필터와 LLM policy pipeline

StockRush

Enterprise Policy RAG

AI Gateway



데이터·AI 파이프라인

원천 이벤트가 검색 답변, 분석 mart, 추천 snapshot으로 변환되는 전 과정을 추적 가능하게 설계했습니다.

원천 수집

변환·저장

계보·품질 확인

CODE AnswerService · CanonicalIngestService · rank_products

- citation/refusal을 포함한 RAG 응답 흐름
- CDC lineage와 행동 기반 추천 snapshot

Enterprise Policy RAG

CDC Data Platform

Fashion Personalization



관측·운영 제어

장애와 품질을 확인할 수 있도록 trace, lag, cost, DLQ, freshness를 API와 화면에 노출했습니다.

signal 기록

API·화면 노출

이상 상태 판단

CODE QueryLogRepository · UsageRollupAggregator · PipelineQualityService

- LLM latency·token·cost와 request trace
- connector lag·quality SLA·batch freshness report

Enterprise Policy RAG

AI Gateway

CDC Data Platform

Fashion Personalization



재현 가능한 검증

동시성, 중복, 장애, 정적 데모를 반복 실행 가능한 smoke, integration, regression으로 고정했습니다.

시나리오 고정

자동 실행

회귀 기준 유지

CODE MvpLiveInfrastructureIT · verify-static-demo · run_portfolio_audit

- Docker Compose와 Testcontainers 인프라 검증
- pytest, offline regression, audit runner 자동화

StockRush

Member Event Consistency

AI Gateway

CDC Data Platform

상태·불변식 정의, 실패 복구 경로, 자동화된 검증을 프로젝트 전반의 구현 기준으로 적용했습니다.

01

StockRush

분산 커머스 / 주문
상태 수렴 / 이벤트
기반 백엔드

ROLE

개인 프로젝트 / 설계,
구현, 검증

SKILLS

분산 트랜잭션
이벤트 수렴
Gateway 보안
장애 복구

STACK

Spring Boot

Kafka

PostgreSQL

Keycloak

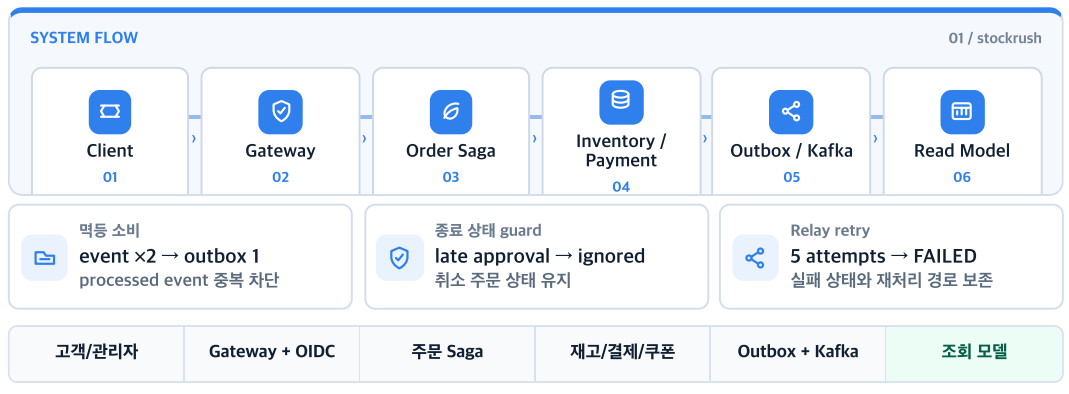
Docker Compose

Android smoke

PROJECT 01 · 분산 커머스

부분 실패 뒤에도 주문 상태를 수렴시키는 이벤트 기반 커머스

한정 재고 주문의 부분 실패와 중복 처리를 Saga, Transactional Outbox, 멍든 소비자로 제어하고 주문·재고·결제·조회 모델의 최종 상태 수렴을 장애 시나리오로 검증했습니다.



01 해결한 문제 부분 실패와 재시도 뒤 주문·재고·결제·조회 상태가 어긋나는 문제
동시 주문, 결제 실패와 지연, Kafka 중단, 이벤트 재처리 상황에서는 주문 상태와 재고 수량, 결제 결과, 조회 모델이 서로 다른 속도로 바뀝니다. 서비스 경계와 복귀 흐름을 코드와 시나리오로 분리해 최종 상태 수렴을 검증했습니다.

02 핵심 설계 판단

01	02	03
Saga orchestration 재고·결제·쿠폰·출고 상태 전이를 주문 서비스에서 조율	Reliable publish DB 변경과 outbox row를 함께 저장해 발행 실패를 재처리	Trust boundary Gateway JWT와 서비스 내부 리소스 소유권 검사를 분리

03 대표 코드 근거

PersistentCreateOrderService.create
.../application/PersistentCreateOrderService.java

```
boolean saved = orderRepository.saveIfAbsent(result.order(),
command.idempotencyKey());
if (!saved) {
    return
    CreateOrderResult.replayed(replayOrder(command.idempotencyKey(),
command.memberId()));
}
outboxEventRepository.save(result.outboxEvent());
```

주문 저장과 outbox 기록을 같은 트랜잭션에 두고 멍든 충돌은 기존 주문 재조회로 수렴시킵니다.
TEST PersistentCreateOrderServiceIntegrationTest

OrderSagaEventHandler.transitionOrder
.../order/application/OrderSagaEventHandler.java

```
where order_id = :orderId
and status not in ('CANCELLED', 'CONFIRMED')

return updated == 1;
```

종료 상태 주문의 추가 전이를 SQL에서 차단해 늦은 이벤트가 취소·확정 주문을 되살리지 못하게 합니다.
TEST OrderSagaEventHandlerIntegrationTest

OutboxRelayService.claimPending
.../infra/outbox/OutboxRelayService.java

```
where status = 'PENDING'
and (next_retry_at is null or next_retry_at <= now())
order by created_at, id
limit :batchSize
for update skip locked
```

재시도 가능 이벤트만 잠금 선점해 여러 relay의 중복 발행을 피하고 소진 실패를 FAILED로 남깁니다.
TEST OutboxRelayServiceIntegrationTest

04 검증 결과

- OrderSagaEventHandlerIntegrationTest가 동일 이벤트 2회 처리에도 outbox 1건만 남도록 고정합니다.
- 취소 완료 뒤 늦게 도착한 결제 승인 이벤트가 주문을 되살리지 않는 상태 guard를 검증합니다.
- OutboxRelayServiceIntegrationTest가 retry 5회 소진 뒤 FAILED 전이와 재처리 상태를 확인합니다.

OUTCOME

한정 판매 커머스의 동시 주문, 결제 실패/지연, 이벤트 발행 실패, 중복 처리, 운영자 보정을 재현 가능한 흐름으로 묶었습니다.

VERIFIED SCOPE

코드와 통합 테스트로 멍든 소비·종료 상태 guard·relay retry를 다뤘으며 실패·다중 리전·운영 규모 부하는 포함하지 않았습니다.

03

Member Event Consistency

동시성 / 업무 불변식 / DB guard / Redis-RabbitMQ 비교

ROLE

2026.05-2026.06 / 개인 프로젝트 / Spring Boot 백엔드, 인프라 비교, 검증 대시보드

SKILLS

동시성 제어

업무 불변식

Redis/RabbitMQ

Testcontainers

STACK

Spring Boot

PostgreSQL

Redis

RabbitMQ

Outbox

React comparison view

PROJECT 03 · 동시성

업무 불변식별로 동시성 전략을 선택하는 회원 이벤트 백엔드

첫 로그인 보상·쿠폰 발급·포인트 차감의 동시성 불변식을 PostgreSQL 제약과 조건부 갱신으로 지키고, Redis lock·RabbitMQ 단일 worker·Outbox 기록을 시나리오별로 비교했습니다.

STRATEGY COMPARISON

03 / member-event-consistency

Scenario API

01

DB Baseline

02

Redis + DB

03

RabbitMQ Lane

04

Outbox

05

Infra Tests

06

최초 보상

8 requests → 1 reward

7 rejected · duplicate 0

포인트 차감

100 - (2 × 60) → 40

1 success · negative 0

쿠폰 용량

8 requests / capacity 3

3 issued · 5 rejected

이벤트 시나리오

불변식 식별

DB guard

Redis/RabbitMQ 비교

Outbox

비교 리포트

01 해결한 문제

동일 회원 이벤트의 중복 보상, 쿠폰 초과 발급, 포인트 음수 잔액
모든 회원 이벤트를 하나의 memberId lock으로 묶으면 hot key 병목이 커지고, 아무 제어 없이 처리하면 중복 보상, 쿠폰 초과 발급, 포인트 음수 잔액, 후속 이벤트 누락이 발생합니다.

02 핵심 설계 판단

01 DB is final guard
unique·check·조건부 갱신·row lock으로 최종 불변식 보호

02 Lock by invariant
Redis lock을 전역 잠금이 아닌 경합 완화 수단으로 사용

03 Queue hot campaigns
RabbitMQ 단일 worker로 hot campaign 직렬화 비교

03 대표 코드 근거

SqlCouponCampaignRepository.issueWithCapacityGuard
---/coupon/SqlCouponCampaignRepository.java

```
where id = ?
  and status = 'ACTIVE'
  and issued_count < capacity
for update
```

캠페인 행 잠금과 용량 조건을 한 SQL 흐름에서 판정해 동시 요청의 초과 발급을 차단합니다.
TEST SqlCouponCampaignRepositoryTest

SqlPointSpendRepository.tryDebit
---/consistency/point/SqlPointSpendRepository.java

```
version = version + 1,
updated_at = now()
where member_id = ?
  and balance >= ?
```

잔액이 지출액 이상일 때만 원자적으로 차감해 동시 요청에서도 음수 잔액을 허용하지 않습니다.
TEST SqlPointSpendRepositoryTest

CouponCampaignRabbitMqWorker.handle
---/web/CouponCampaignRabbitMqWorker.java

```
@RabbitListener(queues =
CouponCampaignRabbitMqConfiguration.COMMAND_QUEUE, concurrency = "1")
public void handle(CouponCampaignRabbitMqCommand command) {
    if (!tracker.isActive(command.operationId())) {
        return;
    }
}
```

단일 consumer로 hot campaign을 직렬화하고 종료된 실행의 메시지를 무시하며 재시도·DLQ 경로를 분리합니다.
TEST MvpLiveInfrastructureIT

04 검증 결과

- FirstLoginRewardDbConcurrencyIT는 동시 요청 8건 중 1건 지급·7건 거절을 assertion으로 고정합니다.
- PointSpendDbConcurrencyIT는 잔액 100에서 60 차감 2건 중 1건만 성공해 잔액 40을 유지합니다.
- MvpLiveInfrastructureIT는 용량 3 캠페인에 8건 요청 시 3건 발급·5건 거절·초과 발급 0을 확인합니다.

OUTCOME

DB guard를 최종 보호 장치로 두고 Redis lock, RabbitMQ lane, Outbox 책임을 업무 불변식별로 분리했습니다.

VERIFIED SCOPE

PostgreSQL·Redis·RabbitMQ 통합 경로를 Testcontainers 시나리오로 고정했으며 운영 규모 부하와 장시간 장애 복구 SLO는 포함하지 않았습니다.

02

Enterprise Policy RAG

AI 백엔드 / RAG /
권한 검색 / 운영 관측

ROLE

2026.05-2026.06 /
개인 프로젝트 / AI
백엔드, 운영 콘솔, 검증
문서화

SKILLS

RAG
권한 필터
근거 답변
운영 관측

STACK

Python API

React/Vite

PostgreSQL/pgvector

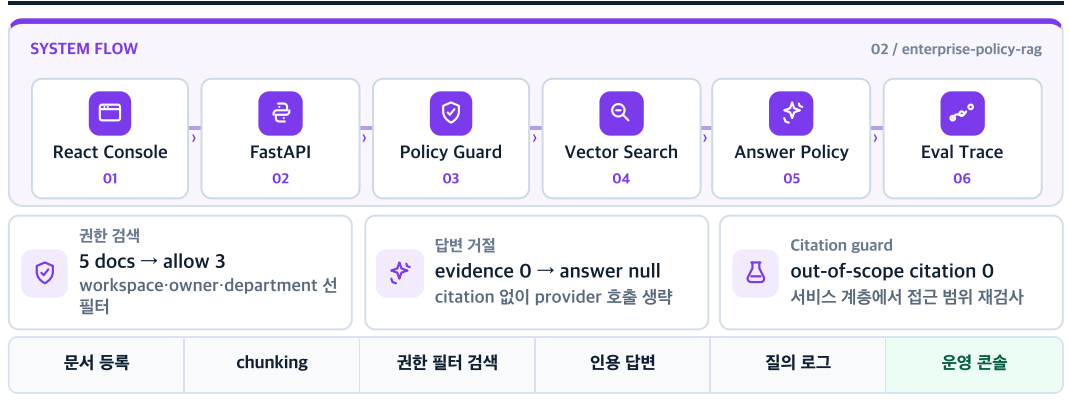
OpenAI 선택 경로

Vercel static demo

PROJECT 02 · AI 백엔드

권한 경계와 근거 응답을 함께 지키는 사내 정책 RAG

권한이 없는 문서의 검색 노출과 근거 없는 답변을 막기 위해 검색 전 권한 필터, 인용·거절 응답, 질의 비용·지연 추적을 갖춘 사내 정책 RAG를 구현했습니다.



01 해결한 문제

권한이 없는 문서가 검색되거나 근거가 부족한 답변이 생성되는 문제
기업 내부 정책 RAG는 검색 품질만으로 부족합니다. 사용자가 볼 수 없는 문서가 검색 후보에 섞이지 않아야 하고, 답변에는 근거가 남아야 하며, 근거가 부족한 경우 거절 응답을 반환해야 합니다.

02 핵심 설계 판단

01

Authorization first

답변 생성 전에 접근 가능한 chunk만 검색 후보로 제한

02

Evidence policy

citation이 없으면 refusal을 반환하고 LLM 호출을 생략

03

Replaceable boundary

provider와 repository를 분리해 기본·외부 경로를 교체

03 대표 코드 근거

RetrievalService.retrieve

app/retrieval.py

```
access_reason = _access_reason(chunk, query)
if access_reason is None:
    continue
```

저장소 필터 뒤에도 워크스페이스·소유자·부서 가시성을 다시 판정해 비인가 결과를 제거합니다.

TEST test_retrieval_allows_public_owner_and_depa...

AnswerService.answer

app/answer.py

```
if not citations:
    return AnswerResponse(
        query=query.query,
        answer=None,
        citations=[],
        refusal_reason="insufficient_evidence",
    )
```

인용 근거가 없으면 provider 호출 없이 명시적인 거절 응답을 반환해 근거 없는 답변을 막습니다.

TEST test_answer_api_refuses_when_no_evidence_is...

run_eval

app/evaluation.py

```
cases = [_run_case(services, request.workspace_id, case) for case in GOLDEN_CASES]
retrieval_hit_rate = _rate(case.retrieval_hit for case in cases)
citation_coverage = _rate(case.citation_covered for case in cases)
```

고정 평가 케이스를 실제 답변 경로로 실행해 검색 적중률과 인용 커버리지를 함께 계산합니다.

TEST test_eval_run_api_returns_retrieval_and_cit...

04 검증 결과

- 권한 검색 테스트에서 문서 5개 중 허용 범위 3개만 반환하고 다른 workspace 결과는 0건으로 고정합니다.
- 근거가 없으면 answer=null, citations=[]을 반환하고 provider 호출을 생략합니다.
- 권한 밖 보안 문서가 검색 결과와 citation에 포함되지 않는 API 시나리오를 검증합니다.

OUTCOME

권한 밖 문서를 검색·인용에서 차단하고, 근거가 없으면 provider 호출 없이 거절 응답을 반환합니다.

VERIFIED SCOPE

기본 경로는 메모리 저장소와 정적 데모로 검증했으며 PostgreSQL/pgvector·OpenAI live 경로와 운영 인증은 선택 검증으로 분리했습니다.

04

AI Gateway

LLM Gateway /
라우팅 / 비용 보호 /
캐시 / 폴백 / 관측

ROLE

2026.06 / 개인
프로젝트 / AI 백엔드
· LLM 플랫폼 설계, 구현,
검증

SKILLS

LLM Gateway
비용 보호
캐시/폴백
WebFlux

STACK

Java 21

Spring Boot WebFlux

Redis

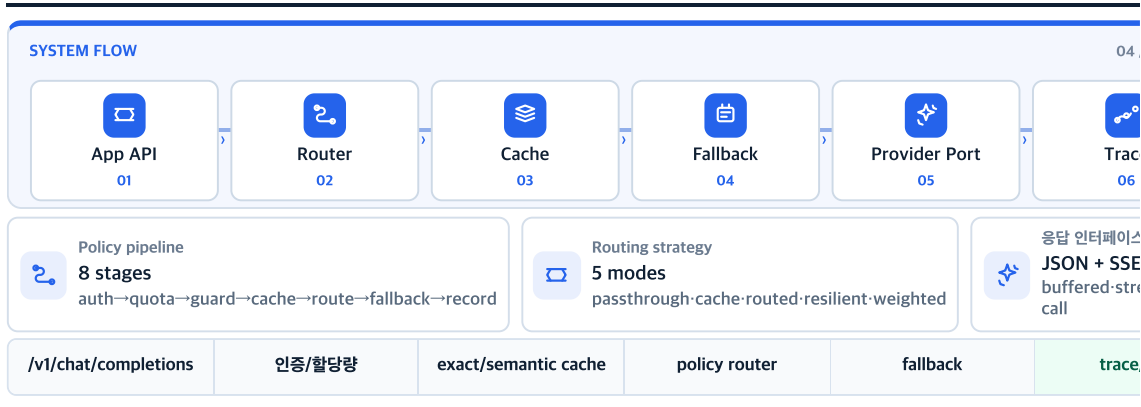
PostgreSQL/pgvector

Testcontainers

PROJECT 04 · LLM Gateway

LLM 호출 정책과 장애 대응을 공통화한 OpenAI 호환 Gateway

OpenAI 호환 WebFlux 진입점에 API key 인증, tenant별 할당량·예산, exact-semantic cache, 정책 routing, circuit breaker·요청 추적을 단일 pipeline으로 구현했습니다.



01 해결한 문제

서비스마다 중복되는 LLM 정책과 provider 장애 대응

LLM 기능을 여러 애플리케이션에 붙이면 provider SDK, 장애 대응, 토큰·비용 예산, 캐시, 보호 규칙, 요청 관측이 서비스마다 중복됩니다. Gateway가 공통 정책과 관측을 처리하도록 구조를 모았습니다.

02 핵심 설계 판단

01	02	
Compatible entry OpenAI 호환 /v1/chat/completions로 클라이언트 변경 최소화	Policy pipeline auth·quota·guard·cache·router·fallback·record 순서 고정	Bounded resilience retry budget·circuit breaker·fallback chain으로 복구 범위 제한

03 대표 코드 근거

GatewayPipeline.execute

.../gateway/api/GatewayPipeline.java

```
QuotaOutcome quota = quotaGuard.evaluate(request, estimatedTokens);
if (quota != QuotaOutcome.ALLOWED) {
    return rejected(request, mode, quota, elapsed(start));
}
```

예산 토큰으로 tenant 할당량을 먼저 검사해 초과 요청을 provider 호출 전에 종료합니다.

TEST RequestLogStoreTest

TwoStageCache.lookup

.../gateway/cache/TwoStageCache.java

```
if (exactLookup.hit()) {
    return exactLookup;
}
return semantic.lookup(request);
```

동일 요청은 exact cache에서 반환하고 miss일 때만 semantic cache로 확장합니다.

TEST TwoStageCacheTest

FallbackChain.dispatch

.../gateway/resilience/FallbackChain.java

```
lastErrorType = RETRY_BUDGET_EXHAUSTED;
events.add(FallbackEvent.failed(attempt, candidate.provider(), candidate.model(),
lastErrorType));
return FallbackResult.ofFailure(attempt, lastErrorType, events);
```

재시도 예산이 소진되면 다음 provider 호출을 차단하고 관측 가능한 실패 이벤트를 남깁니다.

TEST FallbackChainTest

04 검증 결과

- ChatCompletionController 테스트
buffered 응답과 SSE streaming 검증합니다.
- 인증 누락 401, 잘못된 요청 400,
quota-budget 보호 규칙을 API 테스트
고정합니다.
- PolicyRouter와 pipeline 테스트가
routing, tool-call passthrough, c
fallback 순서를 확인합니다.

OUTCOME

앱별 LLM 호출 코드에 흩어질 수 있는 라우팅, 비용 보호, 캐시, 폴백, 관측 기능을 Gateway 계층으로 모았습니다.

VERIFIED SCOPE

기본 검증은 고정 응답 provider와 메모리 저장소를 사용했으며 실제 p
전송·운영 저장소·클라우드 배포는 포함하지 않았습니다.

05

CDC Data
PlatformCDC / Kafka /
Lakehouse / Lineage
/ Replay

ROLE

2026.06 / 개인
프로젝트 / CDC 데이터
플랫폼 백엔드 설계,
구현, 검증

SKILLS

CDC
Kafka 복구
Lineage
Lakehouse

STACK

Spring Boot

PostgreSQL

Debezium

Kafka

Iceberg

MinIO/S3

PROJECT 05 · CDC

변경 이벤트의 중복·재처리·lineage를 추적하는 CDC 플랫폼

PostgreSQL 변경을 Debezium·Kafka로 수집하고, 먹등 ledger와 retry/DLQ/replay를 거쳐
MinIO·Iceberg·mart까지 이어지는 lineage를 검증했습니다.

01 해결한 문제

CDC 중복 replay, sink 실패, source-to-mart 추적 단절

지원자, 공고, 평가, agent task 상태가 계속 변경되는 도메인에서는 운영 API와 분석 테이블 반영 속도가 다를 수 있습니다. source DB 변경의 LSN, offset, event id를 잃지 않고 runtime과 lakehouse 검증까지 이어지도록 구현했습니다.

02 핵심 설계 판단

01

Metadata-derived ID

LSN-offset를 event ID와
replay-lineage 기준으로 사용

02

Separate failure lanes

raw-canonical-retry-DLQ-replay 흐름을
분리

03

Local-first proof

MinIO·Iceberg·Parquet·mart를
로컬에서 반복 검증

03 대표 코드 근거

CanonicalIngestService. ingestRawEnvelope

.../cdcplatform/event/CanonicalIngestService.java

```
DebeziumEnvelope envelope = DebeziumEnvelope.parse(rawEnvelopeJson);  
CanonicalCdcEvent event = CanonicalCdcEvent.fromEnvelope(envelope);  
String sourceOffsetJson = sourceOffsetJson(envelope);  
boolean created = canonicalEventPublisher.recordCanonicalEvent(event,  
sourceOffsetJson);
```

Debezium 원본을 표준 이벤트로 정규화하면서 source offset을 보존하고 ledger 결과로 중복을 판정합니다.

TEST CanonicalIngestServiceTest

ReplayRequestService.requestReplay

.../cdcplatform/replay/ReplayRequestService.java

```
DlqEventRepository.DlqEvent dlqEvent =  
dlqEventRepository.findById(dlqEventId)  
.orElseThrow(() -> new IllegalArgumentException("DLQ event not  
found: " + dlqEventId));  
String requestId = "replay_" + UUID.randomUUID();
```

존재하는 DLQ 이벤트만 replay 대상으로 허용하고 요청 추적용 고유 ID를 생성합니다.

TEST ReplayRequestServiceTest

IcebergJavaEngine.append

.../cdcplatform/tools/IcebergJavaEngine.java

```
DataFile dataFile = writer.toDataFile();  
table.newAppend().appendFile(dataFile).commit();  
table.refresh();  
Snapshot snapshot = table.currentSnapshot();
```

Parquet data file을 Iceberg append로 commit하고 현재 snapshot을 다시 조회해 적재 이력을 남깁니다.

TEST LakehouseSmokeTest

04 검증 결과

- local lineage fixture가 raw event 7건
중 6건을 적용하고 duplicate 1건을
억제합니다.
- applicant-job-evaluation-agent task
4개 source table의 metadata와
lineage key를 보존합니다.
- lineage fixture 2개 중 1개 complete
path를 식별하고 incomplete 경로를
분리해 보고합니다.

OUTCOME

source DB부터 mart까지 LSN, offset, event id, lineage를
추적하고 duplicate replay와 sink failure contamination을
분리합니다.

VERIFIED SCOPE

기본 audit는 fixture 기반 lineage와 제어면을 검증합니다. live
MinIO·engine-backed Iceberg·AWS S3/Athena·Trino 실행은
포함하지 않았습니다.

06

Fashion
Personalization
PlatformPython API / 커머스
개인화 / 추천 batch /
이벤트 처리

ROLE

2026.07 / 개인
프로젝트 / Python
백엔드, 이벤트 처리,
추천 batch, 운영 전환
설계

SKILLS

Python API
개인화 추천
이벤트 처리
Batch workflow

STACK

Python 3.11+

optional FastAPI

pytest

event processing

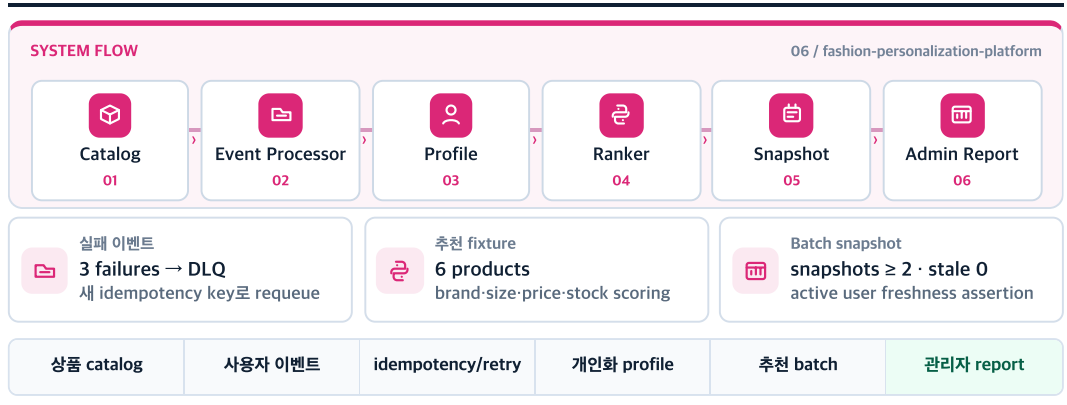
batch workflow

InMemory
repository

PROJECT 06 · Python API

행동 이벤트를 추천 snapshot으로 연결하는 패션 개인화 백엔드

상품 특성과 사용자 행동 이벤트를 개인화 profile과 추천 snapshot으로 변환하고, 중복·실패 이벤트 격리와 freshness 운영 지표를 갖춘 Python 백엔드를 구현했습니다.



01 해결한 문제

중복·실패 이벤트가 개인화 profile과 추천 snapshot을 오염하는 문제

패션 커머스 추천은 브랜드, 카테고리, 태그, 사이즈, 가격대, 재고 같은 catalog feature와 사용자 행동 이벤트가 함께 반영되어야 합니다. 중복 이벤트나 처리 실패가 profile을 오염시키지 않도록 이벤트 안정성, batch snapshot, 관리자 리포트를 하나의 백엔드 흐름으로 분리했습니다.

02 핵심 설계 판단

01

Dependency-free core

외부 설치 없이 domain·service 흐름을 검증

02

Failure isolation

idempotency·retry
budget·DLQ·requeue로 오염 차단

03

Realtime vs batch

profile 갱신과 materialized snapshot 생성을 분리

03 대표 코드 근거

apply_event_to_profile

src/fashion_personalization/recommendation.py

```
selected_size = event.payload.get("size")
if isinstance(selected_size, str) and selected_size in product.sizes:
    _add_weight(profile.size_preferences, selected_size, max(weight, 0.1))
```

선택한 값이 실제 상품 옵션에 있을 때만 사이즈 선호에 반영해 잘못된 프로필 신호를 막습니다.

TEST test_size_preference_contributes_to_ranking

rank_products

src/fashion_personalization/recommendation.py

```
score, reasons = _score_product(profile, product)
recommendations.append(Recommendation(product_id=product.product_id,
score=round(score, 4), reasons=tuple(reasons)))
return sorted(recommendations, key=lambda item: (-item.score, item.product_id))[:limit]
```

점수와 추천 이유를 함께 저장하고 상품 ID tie-break로 결과 순서를 결정적으로 유지합니다.

TEST test_personal_ranking_prioritizes_user_pref...

RecommendationBatchWorkflow.refresh_snapshots

src/fashion_personalization/batch.py

```
generated_at=utc_now(),
source_event_count=profile.event_count,
)
self.store.save_snapshot(snapshot)
```

snapshot에 생성 시각과 반영 이벤트 수를 기록해 결과 최신성과 event watermark를 추적합니다.

TEST test_batch_refresh_writes_materialized_snap...

04 검증 결과

- EventProcessor 테스트가 동일 idempotency key를 한 번만 반영하고 3회 실패 뒤 DLQ로 격리합니다.
- 추천 테스트가 6개 상품 fixture에서 브랜드·사이즈·가격·재고 조건을 deterministic ranking에 반영합니다.
- Batch workflow 테스트가 active user별 snapshot을 2개 이상 만들고 stale snapshot 0을 확인합니다.

OUTCOME

중복 이벤트를 한 번만 반영하고, 3회 실패를 DLQ로 격리하며 추천 snapshot과 freshness 지표를 생성합니다.

VERIFIED SCOPE

검증 구현은 인메모리 저장소를 사용하며 FastAPI는 선택 adapter입니다. PostgreSQL·SQS·EventBridge·S3·AWS 배포는 포함하지 않았습니다.

01

StockRush

분산 커머스 / 주문 상태 수렴 / 이벤트 기반 백엔드

대표 프로젝트

먹통 소비

event x2 → outbox 1

종료 상태 guard

late approval → ignored

Relay retry

5 attempts → FAILED

검증 범위

코드와 통합 테스트로 먹통 소비·종료 상태 guard-relay retry를 다뤘으며 실결제·다중 리전·운영 규모 부하는 포함하지 않았습니다.

Source · Test Strategy · Individual PDF

04

AI Gateway

LLM Gateway / 라우팅 / 비용 보호 / 캐시 / 폴백 / 관측

직무별 확장

Policy pipeline

8 stages

Routing strategy

5 modes

응답 인터페이스

JSON + SSE

검증 범위

기본 검증은 고정 응답 provider와 메모리 저장소를 사용했으며 실제 provider 전송·운영 저장소·클라우드 배포는 포함하지 않았습니다.

Source · Static Demo Verification · Individual PDF

03

Member Event Consistency

동시성 / 업무 불변식 / DB guard / Redis-RabbitMQ 비교

대표 프로젝트

최초 보상

8 requests → 1 reward

포인트 차감

100 - (2 × 60) → 40

쿠폰 용량

8 requests / capacity 3

검증 범위

PostgreSQL·Redis·RabbitMQ 통합 경로를 Testcontainers 시나리오로 고정했으며 운영 규모 부하와 장시간 장애 복구 SLO는 포함하지 않았습니다.

Source · Local Infra README · Individual PDF

05

CDC Data Platform

CDC / Kafka / Lakehouse / Lineage / Replay

직무별 확장

중복 억제 fixture

7 raw → 6 applied

Source coverage

4 source tables

Lineage fixture

2 rows / 1 complete

검증 범위

기본 audit는 fixture 기반 lineage와 제어면을 검증합니다. live MinIO·engine-backed Iceberg·AWS S3/Athena·Trino 실행은 포함하지 않았습니다.

Source · Portfolio One Pager · Individual PDF

02

Enterprise Policy RAG

AI 백엔드 / RAG / 권한 검색 / 운영 관측

대표 프로젝트

권한 검색

5 docs → allow 3

답변 거절

evidence 0 → answer null

Citation guard

out-of-scope citation 0

검증 범위

기본 경로는 메모리 저장소와 정적 데모로 검증했으며 PostgreSQL/pgvector·OpenAI live 경로와 운영 인증은 선택 검증으로 분리했습니다.

Source · Hardening Checklist · Individual PDF

06

Fashion Personalization Platform

Python API / 커머스 개인화 / 추천 batch / 이벤트 처리

직무별 확장

실패 이벤트

3 failures → DLQ

추천 fixture

6 products

Batch snapshot

snapshots ≥ 2 · stale 0

검증 범위

검증 구현은 인메모리 저장소를 사용하며 FastAPI는 선택 adapter입니다. PostgreSQL·SQS/EventBridge·S3·AWS 배포는 포함하지 않았습니다.

Local source · Architecture · Individual PDF

IMPLEMENTED	VERIFIED	OUT OF SCOPE
코드와 설정으로 확인되는 구현 범위	테스트·smoke-장에 시나리오로 반복 확인한 범위	실행 근거가 없어 결과로 표시하지 않은 범위